



ЦЕНТРАЛЬНЫЙ
УНИВЕРСИТЕТ

Оптимизация запросов

SQL И БАЗЫ ДАННЫХ

НЕДЕЛЯ 8

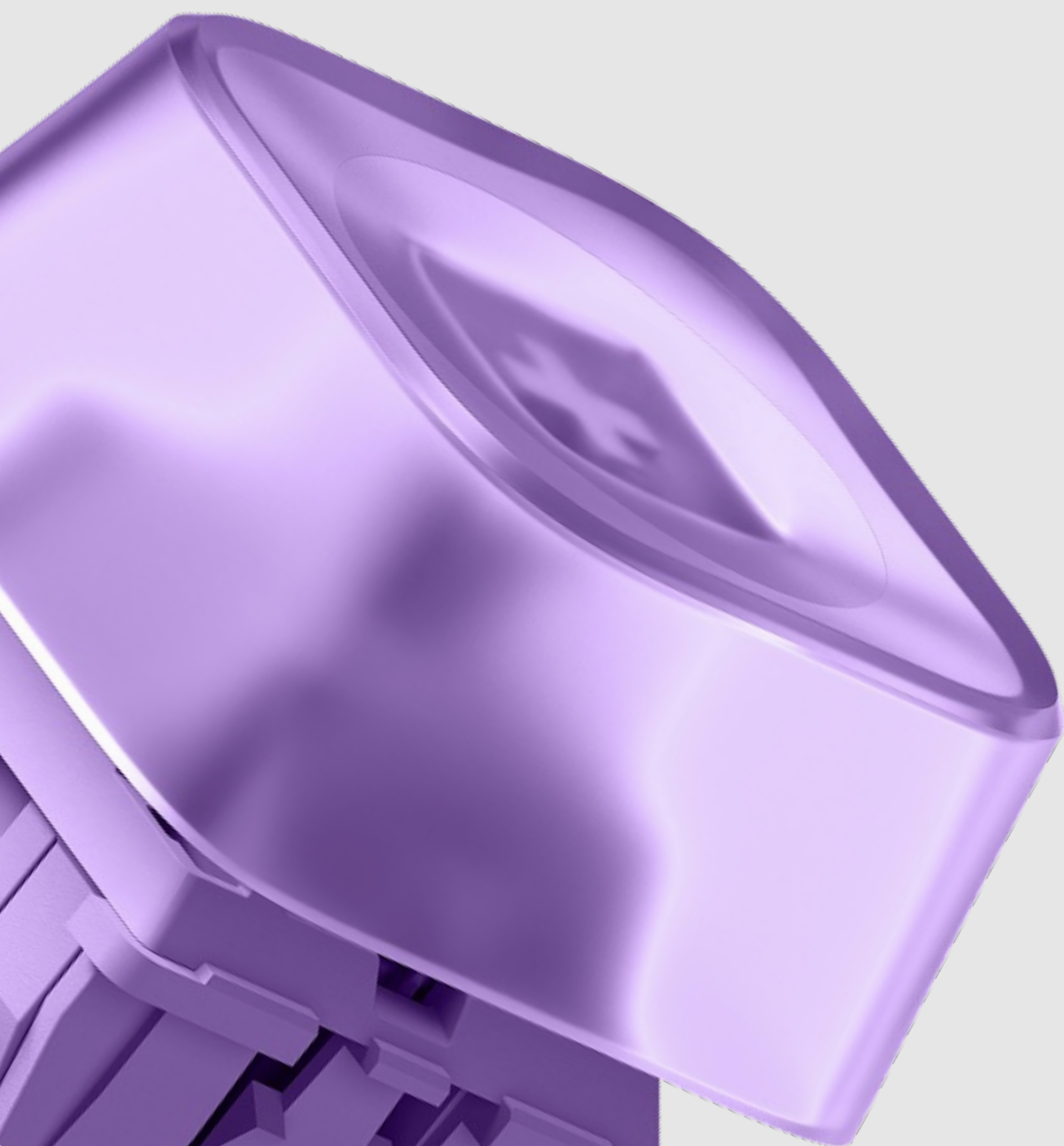
План лекции

→ Введение в оптимизацию

→ План запроса

→ Анализ запросов с помощью explain

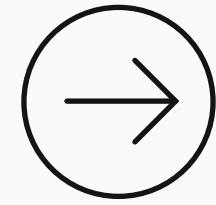
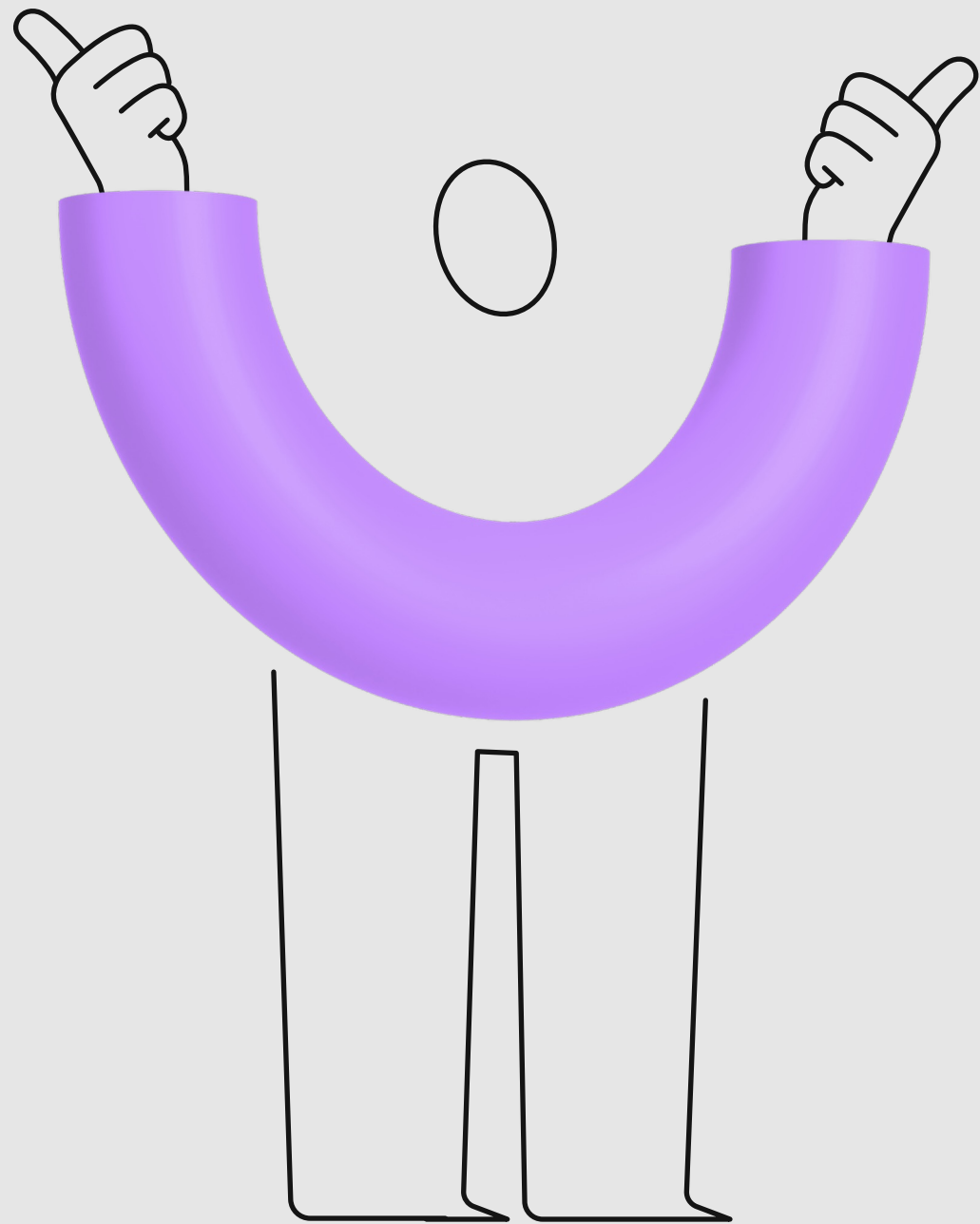
→ Основы оптимизации



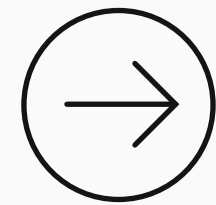


Оптимизация SQL-запросов — это процесс улучшения производительности, который требует глубокого понимания как структуры данных, так и особенностей работы СУБД.

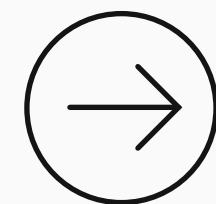
Корректная оптимизация позволяет



Сократить время выполнения запросов



Снизить нагрузку на сервер



Повысить масштабируемость

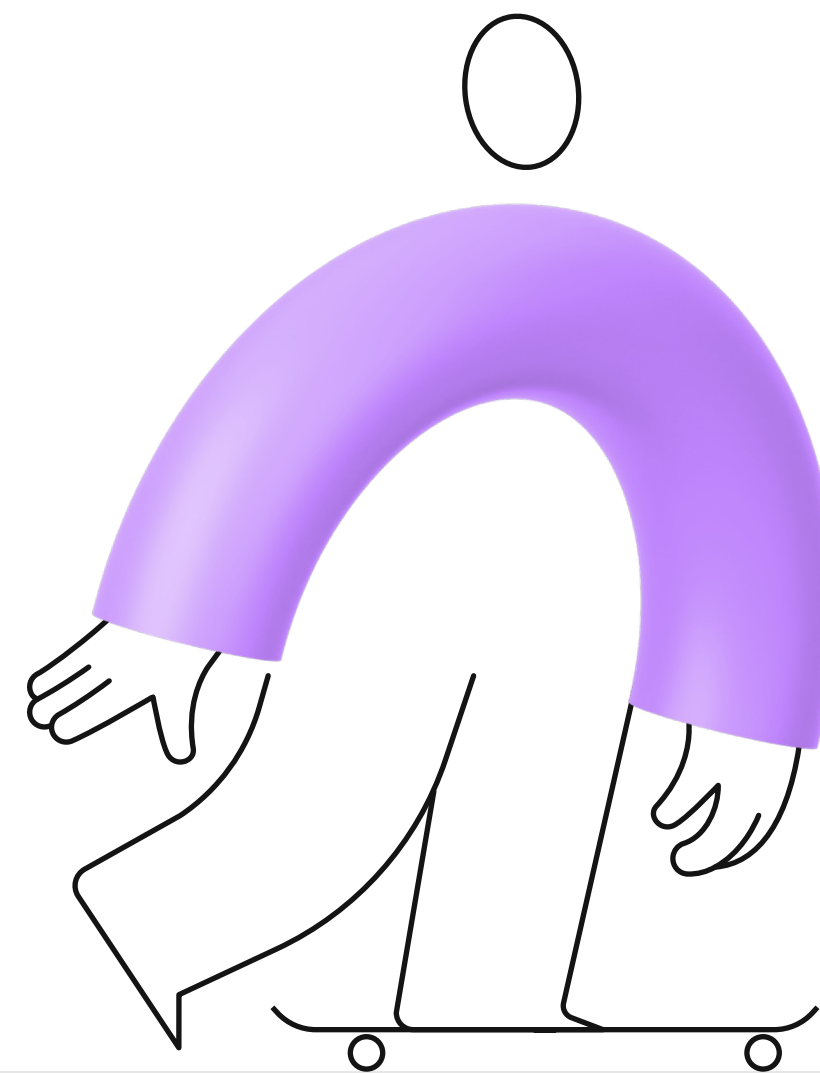


Для понимания оптимизации необходимо разобраться в особенностях и ключевых понятиях, таких как:

- планировщик запросов,
- операции планировщика запросов,
- команды **EXPLAIN** и **EXPLAIN ANALYZE**.

Планировщик запросов

Планировщик запросов (query planner) — это компонент СУБД, который анализирует SQL-запрос, строит возможные планы его выполнения и выбирает наилучший из них на основе «стоимости» выполнения.



Этапы работы планировщика



01

Разбор SQL-запроса (parsing)

02

Анализ и переписывание (rewriting)

03

Генерация возможных планов (planning)

04

Оценка стоимости (cost estimation)

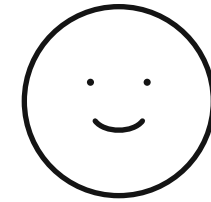
05

Выбор оптимального плана (plan selection)

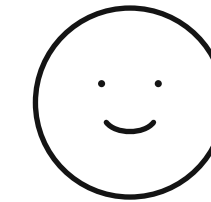
Планировщик запросов



Параллельное выполнение



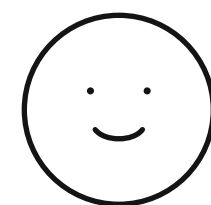
Объём данных в таблицах



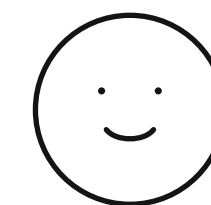
Типы соединений



Кеширование
и буферизация



Наличие индексов



Партиционирование
таблиц

План запроса

Представляет собой подробное описание последовательности шагов, которые СУБД выполняет для реализации SQL-запроса.

План состоит из операций, которые отличаются от привычного нам SQL.

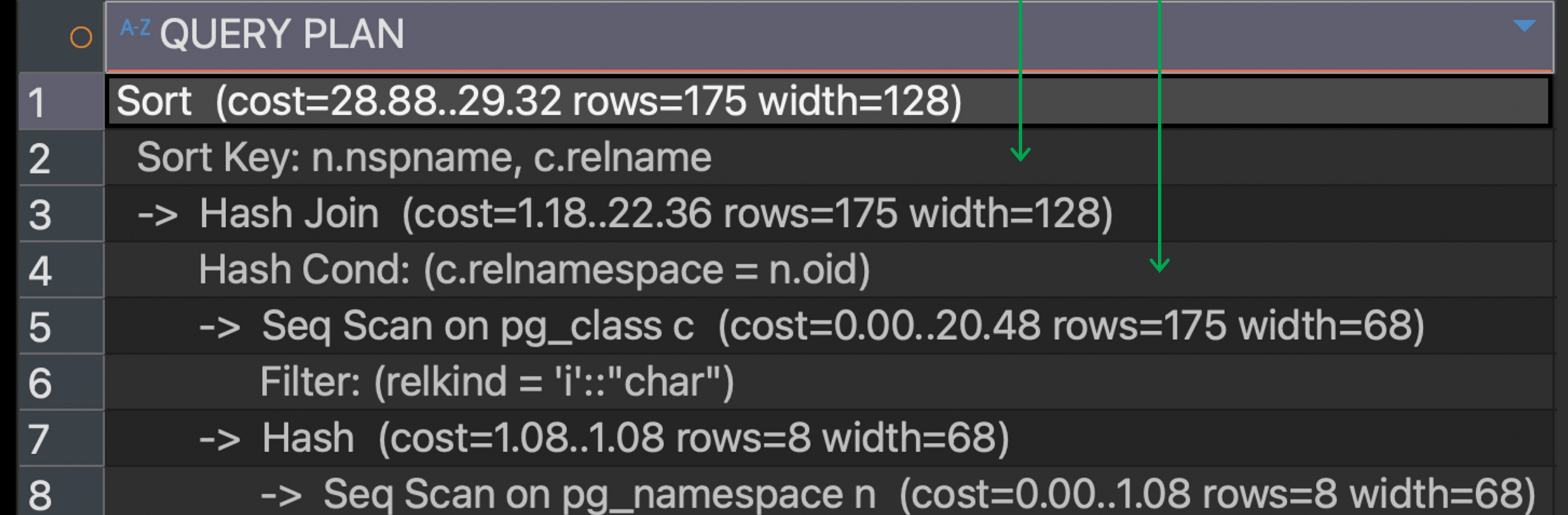
	A-Z QUERY PLAN
1	Sort (cost=28.88..29.32 rows=175 width=128)
2	Sort Key: n.nspname, c.relname
3	-> Hash Join (cost=1.18..22.36 rows=175 width=128)
4	Hash Cond: (c.relnamespace = n.oid)
5	-> Seq Scan on pg_class c (cost=0.00..20.48 rows=175 width=68)
6	Filter: (relkind = 'i'::"char")
7	-> Hash (cost=1.08..1.08 rows=8 width=68)
8	-> Seq Scan on pg_namespace n (cost=0.00..1.08 rows=8 width=68)

План запроса

Представляет собой подробное описание последовательности шагов, которые СУБД выполняет для реализации SQL-запроса.

План состоит из операций, которые отличаются от привычного нам SQL.

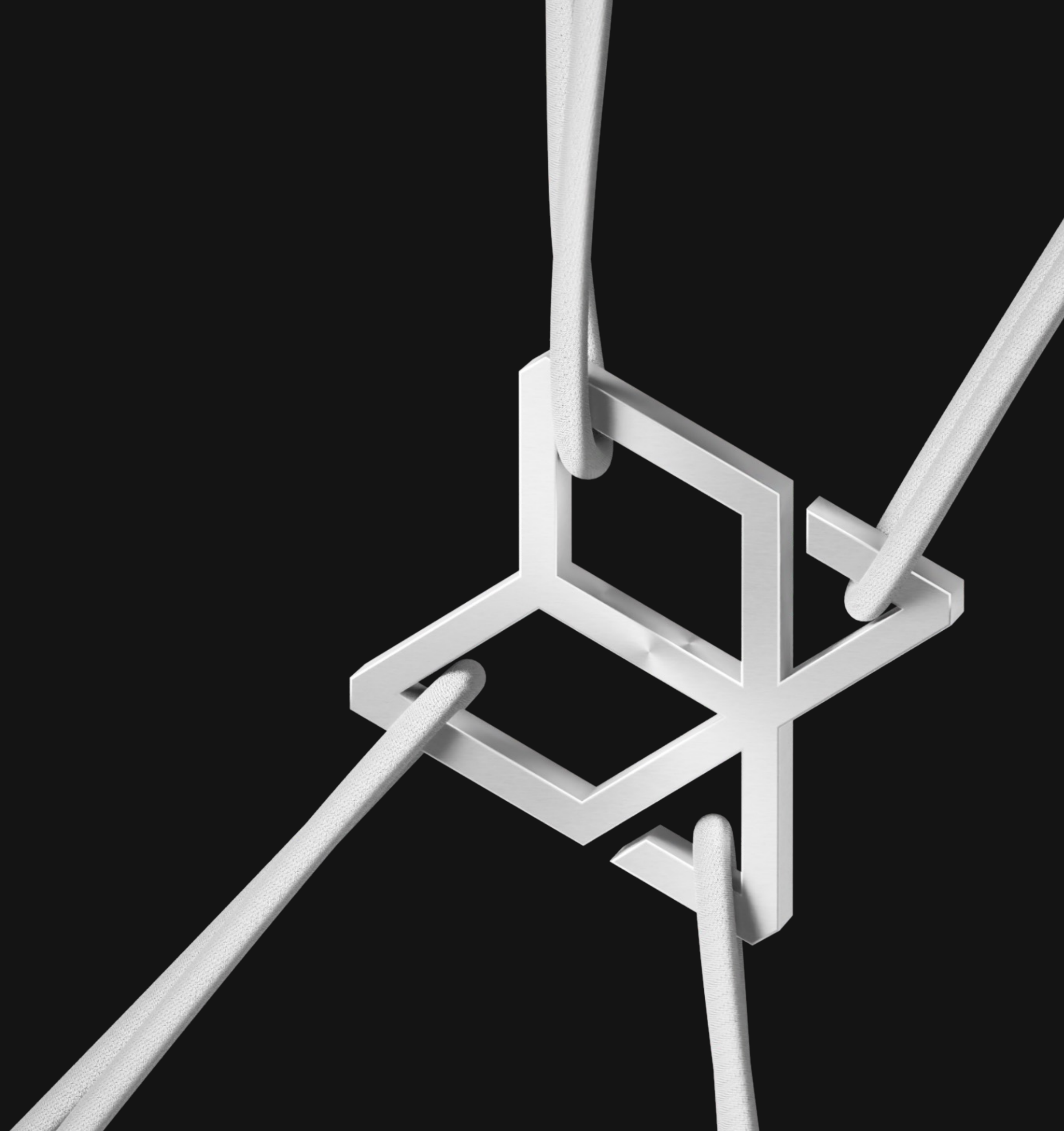
Некоторые операции могут быть выполнены разными способами.



The screenshot shows a query plan with 8 steps. Two green arrows point from the text above to specific steps: one points to step 1 (Sort) and the other points to step 4 (Hash Cond). This illustrates that the same logical operation can be implemented using different physical methods (e.g., Sort vs. Hash join).

	A-Z QUERY PLAN
1	Sort (cost=28.88..29.32 rows=175 width=128)
2	Sort Key: n.nspname, c.relname
3	-> Hash Join (cost=1.18..22.36 rows=175 width=128)
4	Hash Cond: (c.relnamespace = n.oid)
5	-> Seq Scan on pg_class c (cost=0.00..20.48 rows=175 width=68)
6	Filter: (relkind = 'i'::"char")
7	-> Hash (cost=1.08..1.08 rows=8 width=68)
8	-> Seq Scan on pg_namespace n (cost=0.00..1.08 rows=8 width=68)

Основные операции/узлы в плане запроса



Операторы чтения данных

- **Seq Scan** — чтение всех записей в таблице.
- **Index Scan / Index Only Scan** — сканирование по индексу, обращение к таблице за данными.
- **Bitmap Index Scan / Bitmap Heap Scan** — построение битовой карты нужных страниц и их чтение.
- **Tid Scan / Tid Range Scan** — выборка по физическим позициям строк.

Операторы соединения данных

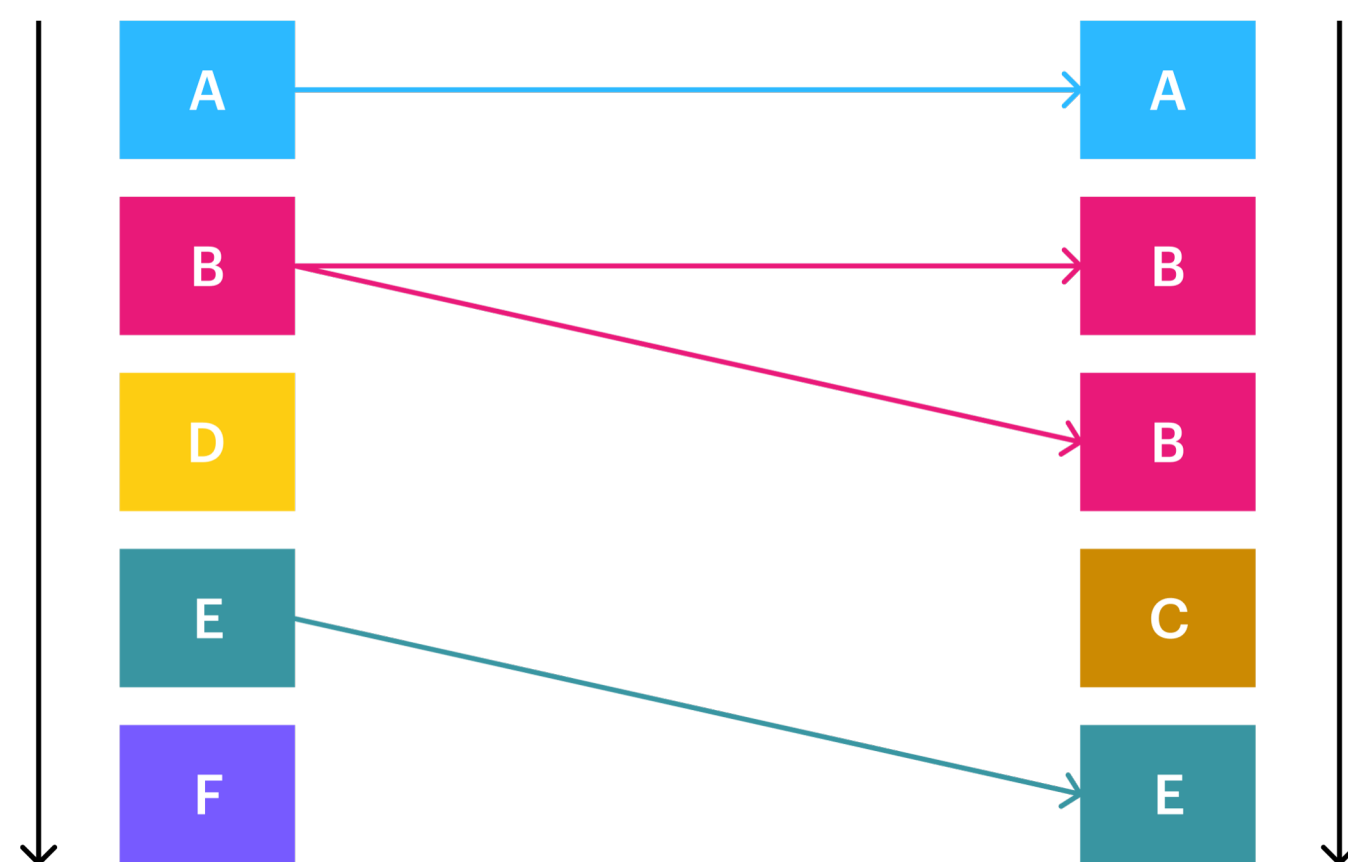
→ Merge Join

→ Nested Loop

→ Hash Join

Merge Join

Соединение отсортированных таблиц

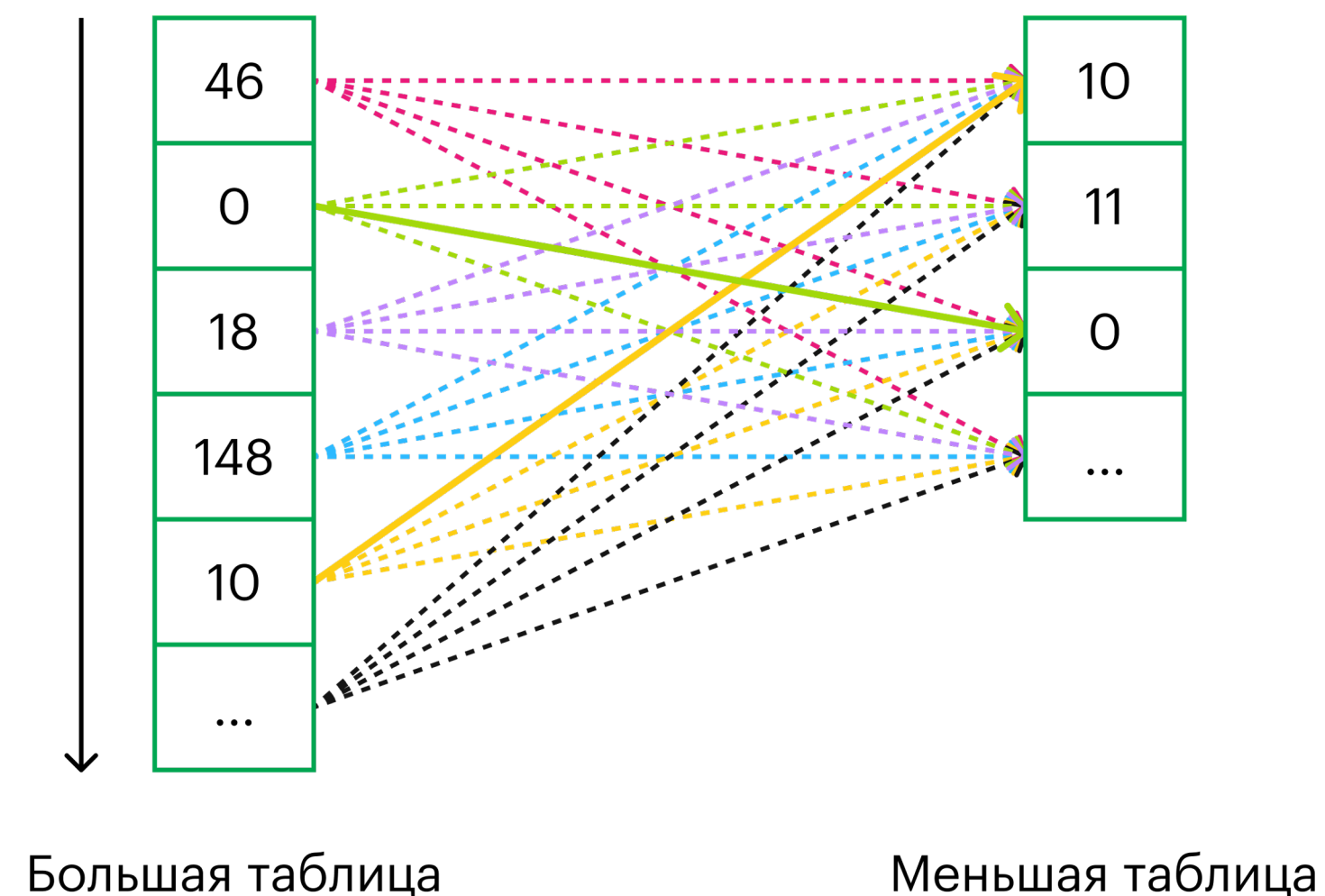


- Для вычисления Merge Join необходима предварительная сортировка данных.
- Соединение происходит за один проход по каждой из таблиц.

На практике Merge Join встречается очень редко.

Nested Loop

Построчный проход по обеим таблицам. Возникает в случае соединения очень маленькой таблицы с большой либо при соединении двух таблиц условием неравенства полей.



- Выполняется проход по строкам большей таблицы.
- Для каждой строки выполняется проход по всем строкам меньшей таблицы.
- В случае выполнения условия соединения соответствующая пара строк попадает в результирующую таблицу.

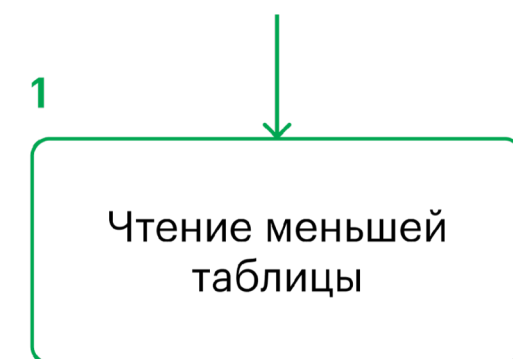
Является крайне неоптимальным способом соединения в случае, когда обе таблицы большие.

Hash Join

Соединение с помощью хеш-таблицы. Возникает в случае соединения двух таблиц по равенству полей.



Алгоритм соединения



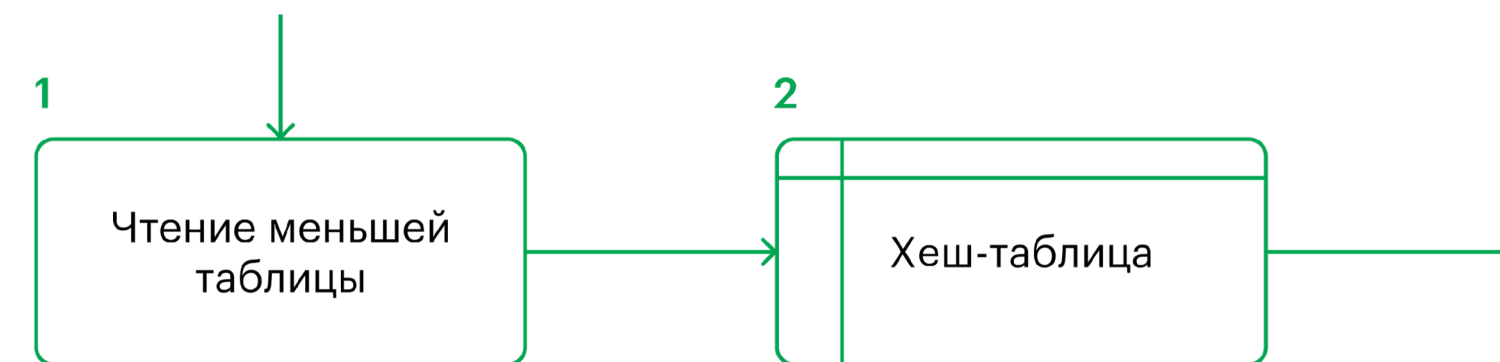
- Происходит чтение меньшей таблицы.

Hash Join

Соединение с помощью хеш-таблицы. Возникает в случае соединения двух таблиц по равенству полей.



Алгоритм соединения



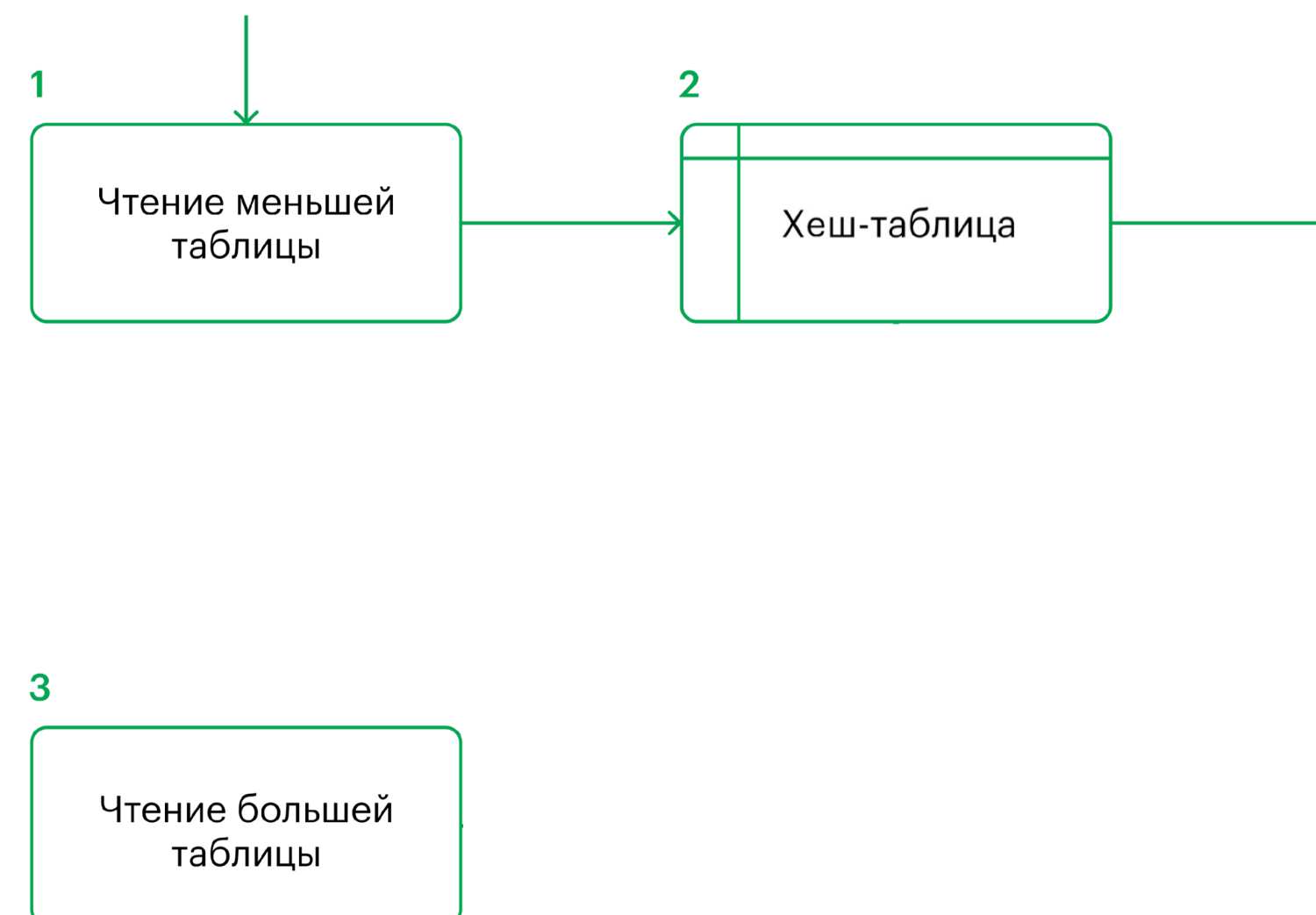
- Происходит чтение меньшей таблицы.
- На основе этой таблицы строится хеш-таблица.

Hash Join

Соединение с помощью хеш-таблицы. Возникает в случае соединения двух таблиц по равенству полей.



Алгоритм соединения



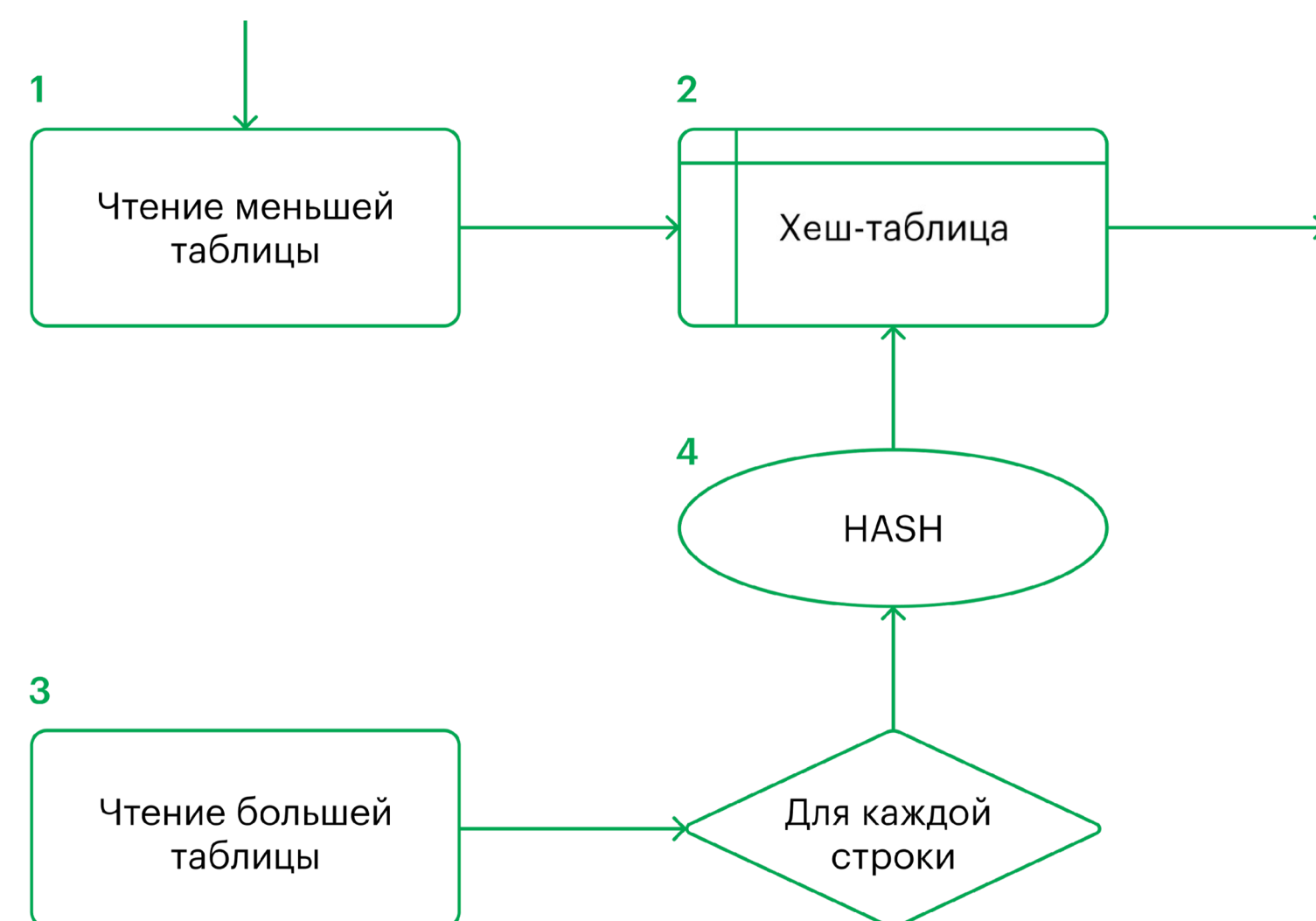
- Происходит чтение меньшей таблицы.
- На основе этой таблицы строится хеш-таблица.
- Затем происходит чтение большей таблицы.

Hash Join

Соединение с помощью хеш-таблицы. Возникает в случае соединения двух таблиц по равенству полей.



Алгоритм соединения



- Происходит чтение меньшей таблицы.
- На основе этой таблицы строится хеш-таблица.
- Затем происходит чтение большей таблицы.
- Для каждой строки большей таблицы вычисляется хеш-функция от ключа соединения и выполняется поиск этого значения в хеш-таблице. В случае успеха соответствующие записи из обеих таблиц добавляются к результату соединения.

Операторы соединения данных

00

Merge Join

01

Hash Join

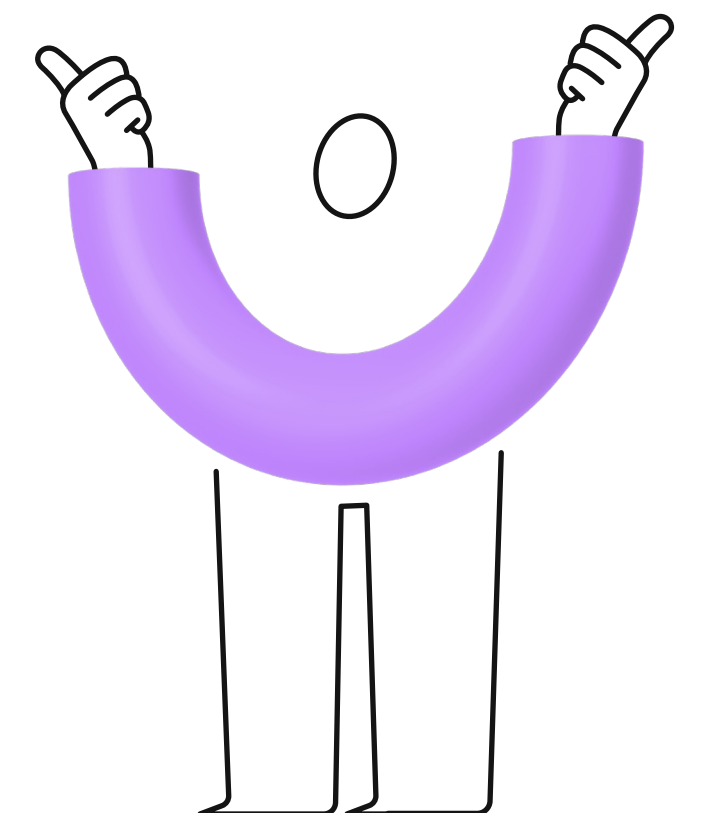
00

Nested Loop

Наиболее предпочтительным и в большинстве случаев оптимальным вариантом является Hash Join.

Merge Join встречается редко, а Nested Loop является дорогим соединением.

Однако бывают ситуации, в которых без построчного прохода по обеим таблицам не обойтись.



Операторы агрегации и группировки

→ **Aggregate / HashAggregate / GroupAggregate** — вычисление агрегатов.

→ **Group** — простая группировка без агрегатных функций.

→ **WindowAgg** — оконные функции (ROW_NUMBER(), RANK() и тому подобное).

Операторы сортировки и ограничений

→ **Sort / Incremental Sort** — полная или поэтапная сортировка.

→ **Limit** — применение LIMIT/OFFSET.

→ **Filter** — принимает на вход поток строк и проверяет условие на каждой строке (предикат WHERE).

Операторы связанные с параллелизмом

→ **Gather / Gather Merge** — сбор и объединение результатов от параллельных воркеров.

→ **Parallel Seq Scan, Parallel Index Scan** и другие — распределение чтения по воркерам.

Вспомогательные операторы

→ **Materialize** — кэширование промежуточного результата в памяти.

→ **Unique** — удаление дубликатов.

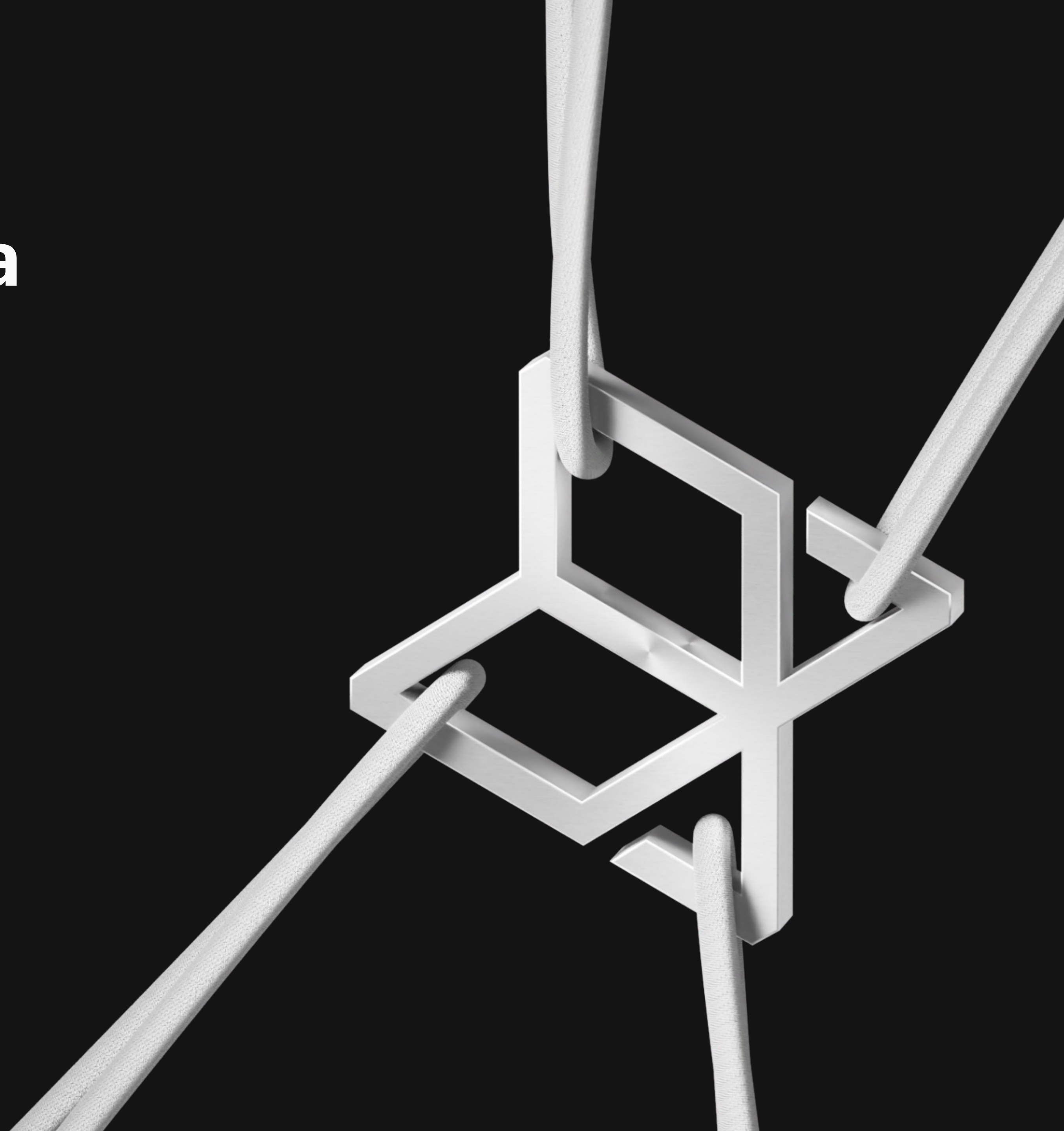
→ **SetOp** — реализация операций множеств (UNION, INTERSECT, EXCEPT).

→ **Partition Selection** — служебный узел, который определяет, какие партиции нужно читать.

Функции узлов

- **Модульность** — каждый узел решает отдельную задачу (чтение, фильтрация, соединение и т. д.).
- **Оптимизация** — оптимизатор может менять порядок и метод конкретного узла для достижения максимальной по его мнению производительности.
- **Диагностика** — по дереву узлов проще понять, где в запросе находится узкое место по производительности.

Чтение плана запроса





План запроса можно посмотреть
с помощью двух основных команд:

- EXPLAIN,
- EXPLAIN ANALYZE.

EXPLAIN

Выводит предполагаемый план выполнения запроса без запуска самого запроса.

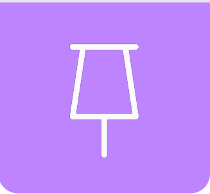


A-Z QUERY PLAN	
1	Sort (cost=28.88..29.32 rows=175 width=128)
2	Sort Key: n.nspname, c.relname
3	-> Hash Join (cost=1.18..22.36 rows=175 width=128)
4	Hash Cond: (c.relnamespace = n.oid)
5	-> Seq Scan on pg_class c (cost=0.00..20.48 rows=175 width=68)
6	Filter: (relkind = 'i':"char")
7	-> Hash (cost=1.08..1.08 rows=8 width=68)
8	-> Seq Scan on pg_namespace n (cost=0.00..1.08 rows=8 width=68)

Метрики:

- cost — стоимость выполнения операции;
- rows — итоговое количество строк;
- width — ширина выходной таблицы в байтах.

EXPLAIN ANALYZE



Выводит фактический план выполнения запроса после запуска самого запроса.

QUERY PLAN

1

Sort (cost=29.24..29.69 rows=180 width=128) (actual time=0.364..0.380 rows=180 loops=1)

2

Sort Key: n.nspname, c.relname

3

Sort Method: quicksort Memory: 54kB

4

-> Hash Join (cost=1.18..22.50 rows=180 width=128) (actual time=0.046..0.233 rows=180 loops=1)

5

Hash Cond: (c.relnamespace = n.oid)

6

-> Seq Scan on pg_class c (cost=0.00..20.60 rows=180 width=68) (actual time=0.011..0.144 rows=180 loops=1)

7

Filter: (relkind = 'i'::"char")

8

Rows Removed by Filter: 268

9

-> Hash (cost=1.08..1.08 rows=8 width=68) (actual time=0.025..0.026 rows=28 loops=1)

10

Buckets: 1024 Batches: 1 Memory Usage: 11kB

11

-> Seq Scan on pg_namespace n (cost=0.00..1.08 rows=8 width=68) (actual time=0.006..0.012 rows=28 loops=1)

12

Planning Time: 0.251 ms

13

Execution Time: 0.425 ms

- Метрики:
- итоговое время в миллисекундах;
 - время в миллисекундах на каждую операцию;
 - объём памяти, который потребовался во время операции;
 - прочая сопроводительная информация.

Статистика

Статистика — это информация о таблицах, которой оперирует оптимизатор для построения оптимального плана запроса. В статистике содержатся информация об объёме таблицы (количестве строк и объёме занимаемой памяти) и различная информация о каждом поле (количестве null-значений, уникальных и наиболее часто встречающихся значений и т. д.).



Принудительный сбор статистики

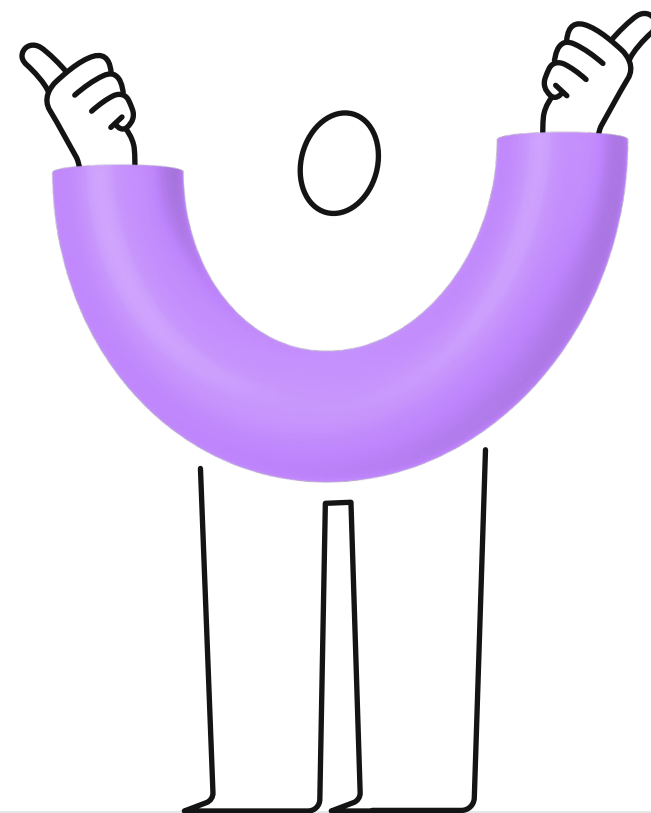
```
ANALYZE [VERBOSE] table_name (field1 [, field2, ...]);
```

VACUUM и AUTOVACUUM

Отчистка таблицы от «мёртвых» строк

```
VACUUM table_name;
```

AUTOVACUUM — в автоматическом режиме занимается удалением данных, а также сбором статистики. Обычно запускается раз в день для всей БД.



Параметры конфигурации PostgreSQL

Параметры управления памятью:

- `shared_buffers` — объём памяти для буферного кеша PostgreSQL;
- `work_mem` — память на одну операцию сортировки/хеширования в запросе и другие.

Параметры конфигурации PostgreSQL

Параметры управления памятью:

- `shared_buffers` — объём памяти для буферного кеша PostgreSQL;
- `work_mem` — память на одну операцию сортировки/хеширования в запросе и другие.

Параметры параллелизма:

- `sharedmax_worker_processes` — общее число вспомогательных процессов;
- `max_parallel_workers` — максимум воркеров для параллельных запросов в кластере.

Параметры конфигурации PostgreSQL

Параметры управления памятью:

- `shared_buffers` — объём памяти для буферного кеша PostgreSQL;
- `work_mem` — память на одну операцию сортировки/хеширования в запросе и другие.

Параметры параллелизма:

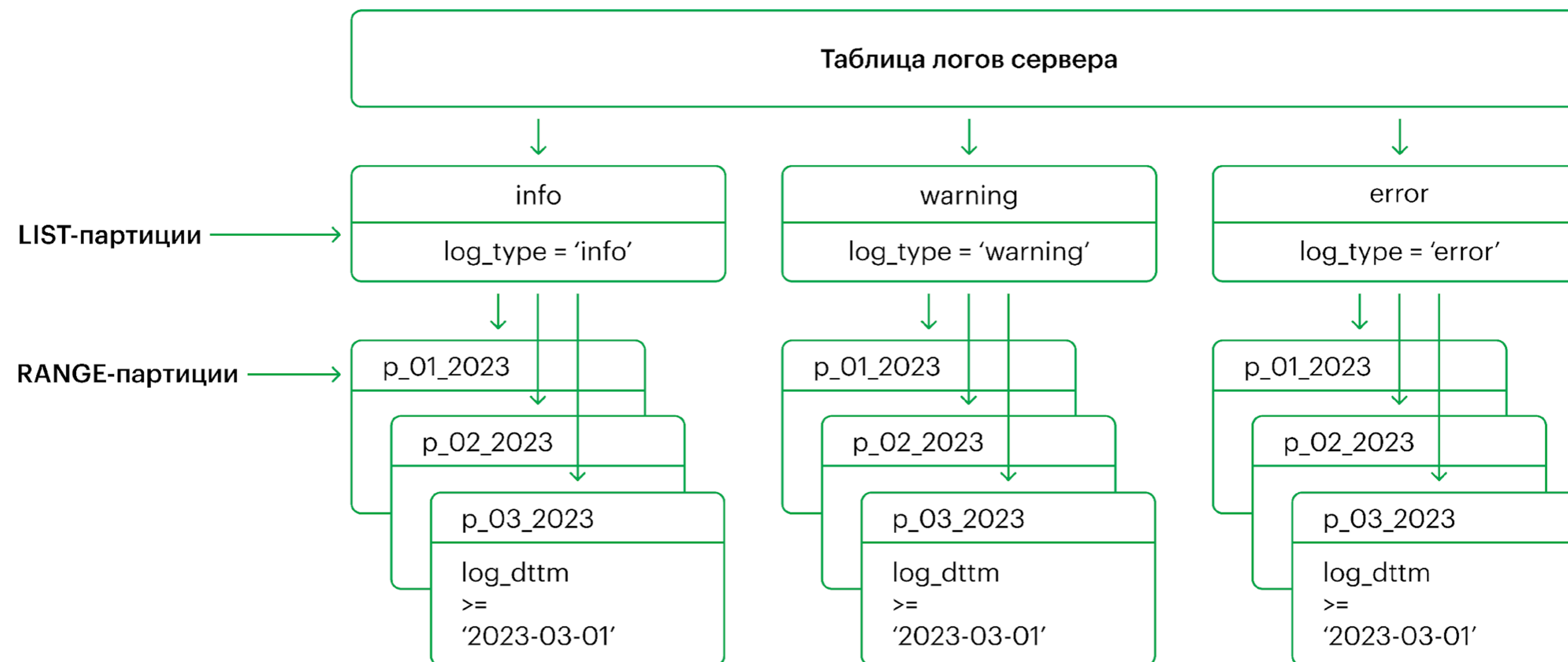
- `sharedmax_worker_processes` — общее число вспомогательных процессов;
- `max_parallel_workers` — максимум воркеров для параллельных запросов в кластере.

Прочие параметры:

- `autovacuum_naptime` — пауза между циклами `autovacuum`;
- `effective_io_concurrency` — максимальное число одновременных асинхронных операций чтения/записи;
- `max_connections` — максимальное число подключений.

Партиционирование (partitioning)

Партиционирование (partitioning) — логическое разбиение больших таблиц на небольшие подтаблицы, называемые партициями (partitions).



Виды партиционирования

by range — по диапазону значений (чаще всего временному).

```
CREATE TABLE sales (  
    id BIGSERIAL,  
    sale_date DATE NOT NULL,  
    amount DECIMAL(12,2),  
    customer_id INT  
)  
PARTITION BY RANGE (sale_date);  
  
CREATE TABLE sales_2024_q1 PARTITION OF sales  
    FOR VALUES FROM ('2024-01-01') TO ('2024-04-01');  
CREATE TABLE sales_2024_q2 PARTITION OF sales  
    FOR VALUES FROM ('2024-04-01') TO ('2024-07-01');
```

Виды партиционирования

by list — по значениям.

```
CREATE TABLE customers (  
    id SERIAL,  
    name TEXT,  
    region TEXT NOT NULL  
)  
PARTITION BY LIST (region);
```

```
CREATE TABLE customers_north PARTITION OF customers  
    FOR VALUES IN ('North', 'Northeast');  
CREATE TABLE customers_south PARTITION OF customers  
    FOR VALUES IN ('South', 'Southeast');
```


Виды партиционирования

by hash — по хешу.

```
CREATE TABLE orders (  
    id BIGSERIAL PRIMARY KEY,  
    cust_id INT NOT NULL,  
    total NUMERIC(10,2),  
    order_ts TIMESTAMP  
)  
PARTITION BY HASH (cust_id);  
  
CREATE TABLE orders_p0 PARTITION OF orders  
    FOR VALUES WITH (MODULUS 4, REMAINDER 0);  
CREATE TABLE orders_p1 PARTITION OF orders  
    FOR VALUES WITH (MODULUS 4, REMAINDER 1);  
CREATE TABLE orders_p2 PARTITION OF orders  
    FOR VALUES WITH (MODULUS 4, REMAINDER 2);  
CREATE TABLE orders_p3 PARTITION OF orders  
    FOR VALUES WITH (MODULUS 4, REMAINDER 3);
```

Виды партиционирования

default — хранит всё, что не попало в остальные партиии.

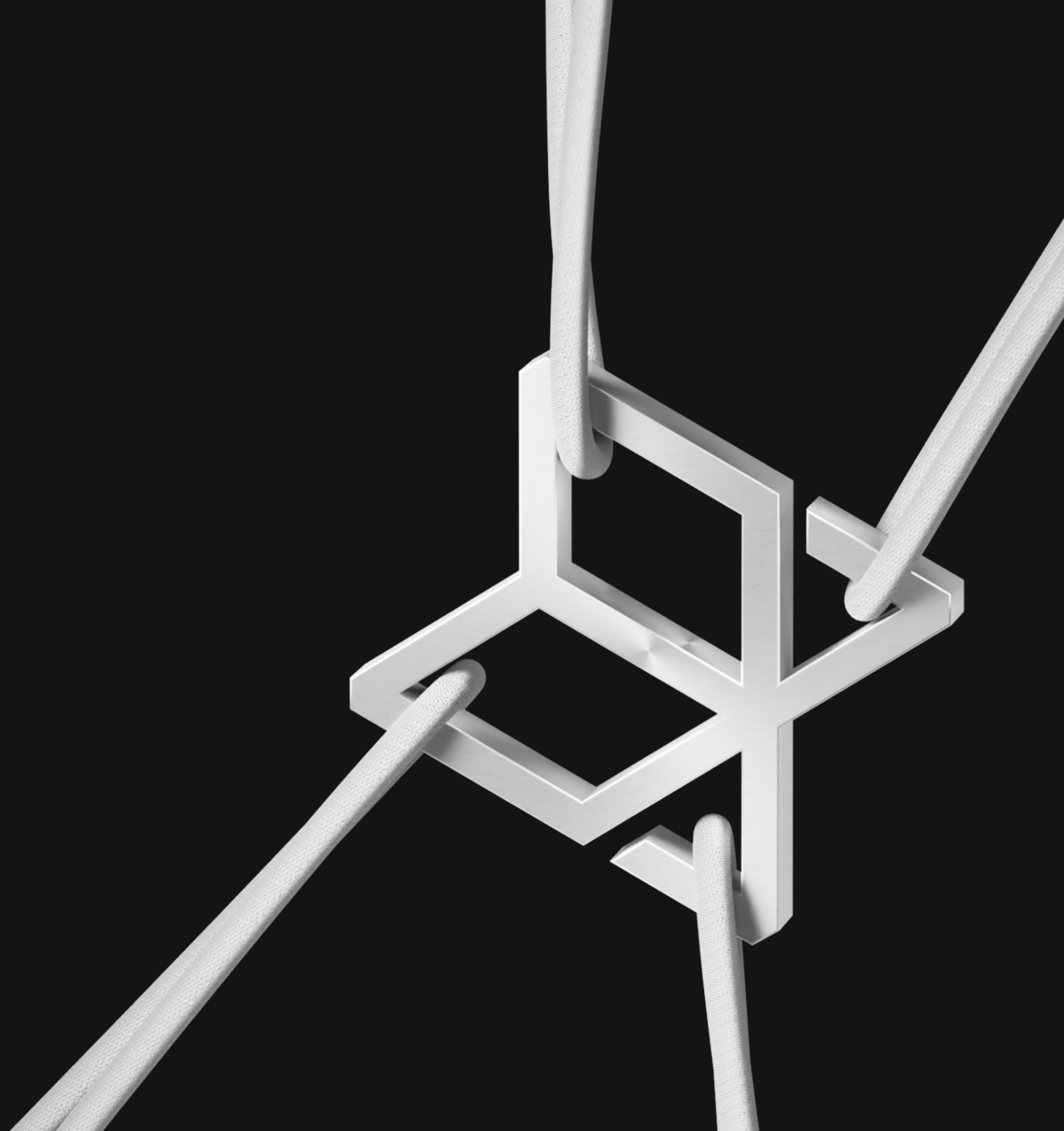
```
CREATE TABLE events_default PARTITION OF events  
    DEFAULT;
```

Дефолтная партиция позволяет:

- избегать ошибок при неожиданных значениях;
- собирать «лишние» или редко встречающиеся данные в одном месте;
- упрощать миграцию и реорганизацию схемы: новые диапазоны можно добавить позже, а промежуточные данные будут аккумулироваться в DEFAULT.



ОСНОВЫ ОПТИМИЗАЦИИ



Рекомендации

1. Использовать **EXPLAIN** перед запуском запроса.
2. Использовать **EXPLAIN ANALYZE** для анализа регулярных запросов.

Рекомендации

1. Использовать **EXPLAIN** перед запуском запроса.
2. Использовать **EXPLAIN ANALYZE** для анализа регулярных запросов.
3. Использовать условие на поле партиционирования, если оно есть.
4. Использовать условие на поле с индексом, если оно есть.

Рекомендации

1. Использовать **EXPLAIN** перед запуском запроса.
2. Использовать **EXPLAIN ANALYZE** для анализа регулярных запросов.
3. Использовать условие на поле партиционирования, если оно есть.
4. Использовать условие на поле с индексом, если оно есть.
5. Создавать временные промежуточные таблицы, если это необходимо.

Рекомендации

1. Использовать **EXPLAIN** перед запуском запроса.
2. Использовать **EXPLAIN ANALYZE** для анализа регулярных запросов.
3. Использовать условие на поле партиционирования, если оно есть.
4. Использовать условие на поле с индексом, если оно есть.
5. Создавать временные промежуточные таблицы, если это необходимо.
6. Не использовать **OR**, **IN**, **LIKE** в условиях соединения **JOIN**.

Рекомендации

1. Использовать **EXPLAIN** перед запуском запроса.
2. Использовать **EXPLAIN ANALYZE** для анализа регулярных запросов.
3. Использовать условие на поле партиционирования, если оно есть.
4. Использовать условие на поле с индексом, если оно есть.
5. Создавать временные промежуточные таблицы, если это необходимо.
6. Не использовать **OR**, **IN**, **LIKE** в условиях соединения **JOIN**.
7. Перед **JOIN** по максимуму использовать фильтры.

Рекомендации

1. Использовать **EXPLAIN** перед запуском запроса.
2. Использовать **EXPLAIN ANALYZE** для анализа регулярных запросов.
3. Использовать условие на поле партиционирования, если оно есть.
4. Использовать условие на поле с индексом, если оно есть.
5. Создавать временные промежуточные таблицы, если это необходимо.
6. Не использовать **OR**, **IN**, **LIKE** в условиях соединения **JOIN**.
7. Перед **JOIN** по максимуму использовать фильтры.
8. Не использовать **SELECT ***.

Рекомендации

1. Использовать **EXPLAIN** перед запуском запроса.
2. Использовать **EXPLAIN ANALYZE** для анализа регулярных запросов.
3. Использовать условие на поле партиционирования, если оно есть.
4. Использовать условие на поле с индексом, если оно есть.
5. Создавать временные промежуточные таблицы, если это необходимо.
6. Не использовать **OR**, **IN**, **LIKE** в условиях соединения **JOIN**.
7. Перед **JOIN** по максимуму использовать фильтры.
8. Не использовать **SELECT ***.
9. Избегать операции **ORDER BY**.

Рекомендации

1. Использовать **EXPLAIN** перед запуском запроса.
2. Использовать **EXPLAIN ANALYZE** для анализа регулярных запросов.
3. Использовать условие на поле партиционирования, если оно есть.
4. Использовать условие на поле с индексом, если оно есть.
5. Создавать временные промежуточные таблицы, если это необходимо.
6. Не использовать **OR**, **IN**, **LIKE** в условиях соединения **JOIN**.
7. Перед **JOIN** по максимуму использовать фильтры.
8. Не использовать **SELECT ***.
9. Избегать операции **ORDER BY**.
10. Избегать операции **DISTINCT**.

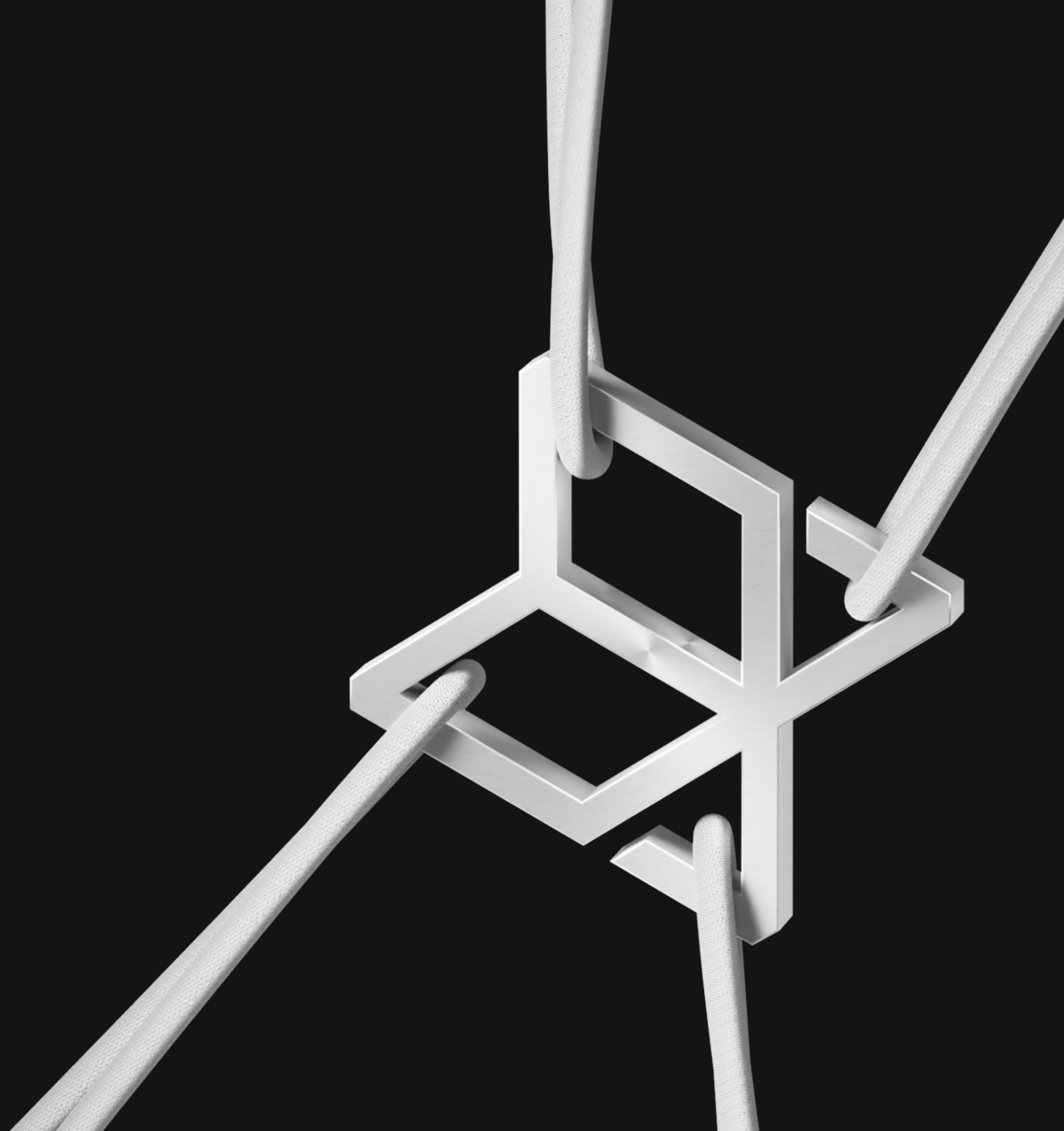
Рекомендации

1. Использовать **EXPLAIN** перед запуском запроса.
2. Использовать **EXPLAIN ANALYZE** для анализа регулярных запросов.
3. Использовать условие на поле партиционирования, если оно есть.
4. Использовать условие на поле с индексом, если оно есть.
5. Создавать временные промежуточные таблицы, если это необходимо.
6. Не использовать **OR**, **IN**, **LIKE** в условиях соединения **JOIN**.
7. Перед **JOIN** по максимуму использовать фильтры.
8. Не использовать **SELECT ***.
9. Избегать операции **ORDER BY**.
10. Избегать операции **DISTINCT**.
11. Избегать операции **UNION**.

Рекомендации

1. Использовать **EXPLAIN** перед запуском запроса.
2. Использовать **EXPLAIN ANALYZE** для анализа регулярных запросов.
3. Использовать условие на поле партиционирования, если оно есть.
4. Использовать условие на поле с индексом, если оно есть.
5. Создавать временные промежуточные таблицы, если это необходимо.
6. Не использовать **OR**, **IN**, **LIKE** в условиях соединения **JOIN**.
7. Перед **JOIN** по максимуму использовать фильтры.
8. Не использовать **SELECT ***.
9. Избегать операции **ORDER BY**.
10. Избегать операции **DISTINCT**.
11. Избегать операции **UNION**.
12. Избегать подзапросов.

Прочие ошибки в запросах



Использование OR в условиях фильтрации

Условия с **OR** могут предотвратить использование индексов и партиций, приводя к полному сканированию таблицы.



Использование OR в условиях фильтрации

Условия с OR могут предотвратить использование индексов и партиций, приводя к полному сканированию таблицы.



```
SELECT *  
FROM my_table  
WHERE col1 = 'A' OR col2 = 'B';
```


Использование OR в условиях фильтрации

Условия с OR могут предотвратить использование индексов и партиций, приводя к полному сканированию таблицы.



```
SELECT *  
FROM my_table  
WHERE col1 = 'A' OR col2 = 'B';
```

Перепишите запрос с использованием UNION ALL.

```
SELECT * FROM my_table WHERE col1 = 'A'  
UNION ALL  
SELECT * FROM my_table WHERE col2 =  
= 'B';
```

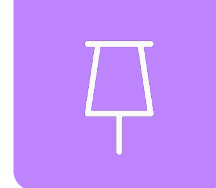
Функции в условиях JOIN

Функции в условиях JOIN,
такие как `upper()` или `substring()`,
могут предотвратить использование
индексов и партиций.



Функции в условиях JOIN

Функции в условиях JOIN, такие как `upper()` или `substring()`, могут предотвратить использование индексов и партиций.



Неоптимальный запрос

```
SELECT *  
FROM table1 t1  
      JOIN table2 t2  
      ON t1.col = upper(t2.col);
```

Функции в условиях JOIN

Функции в условиях JOIN, такие как `upper()` или `substring()`, могут предотвратить использование индексов и партиций.



Неоптимальный запрос

```
SELECT *
FROM table1 t1
JOIN table2 t2
ON t1.col = upper(t2.col);
```

Варианты оптимизации

```
CREATE TEMP TABLE tmp AS
SELECT *, UPPER(col) AS upper_col
FROM table2;
```

```
SELECT *
FROM table1 t1
JOIN tmp t2
ON t1.col = t2.upper_col;
```

Функции в условиях JOIN

Функции в условиях JOIN, такие как `upper()` или `substring()`, могут предотвратить использование индексов и партиций.



Неоптимальный запрос

```
SELECT *
FROM table1 t1
JOIN table2 t2
ON t1.col = upper(t2.col);
```

Варианты оптимизации

```
CREATE TEMP TABLE tmp AS
SELECT *, UPPER(col) AS upper_col
FROM table2;
```

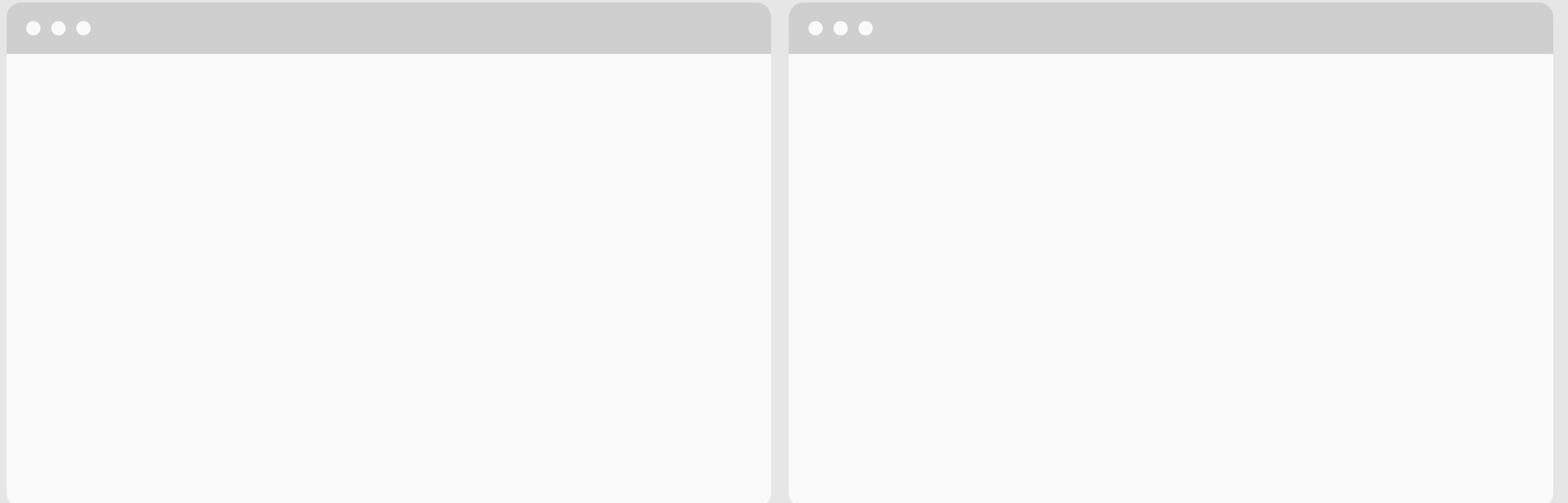
```
SELECT *
FROM table1 t1
JOIN tmp t2
ON t1.col = t2.upper_col;
```

```
ALTER TABLE table2
ADD COLUMN upper_col VARCHAR(100);
UPDATE table2 SET upper_col = UPPER(col);
```

```
SELECT *
FROM table1 t1
JOIN table2 t2
ON t1.col = t2.upper_col;
```


Функции в условиях WHERE

Применение любых функций к полям при фильтрации может привести к тому, что оптимизатор перестанет учитывать индексы и партиции.



Функции в условиях WHERE

Применение любых функций к полям при фильтрации может привести к тому, что оптимизатор перестанет учитывать индексы и партиции.



Неоптимальный запрос

```
SELECT *  
FROM table1 t1  
WHERE LEFT(phone_number, 2) = '+7'
```

Функции в условиях WHERE

Применение любых функций к полям при фильтрации может привести к тому, что оптимизатор перестанет учитывать индексы и партиции.



Неоптимальный запрос

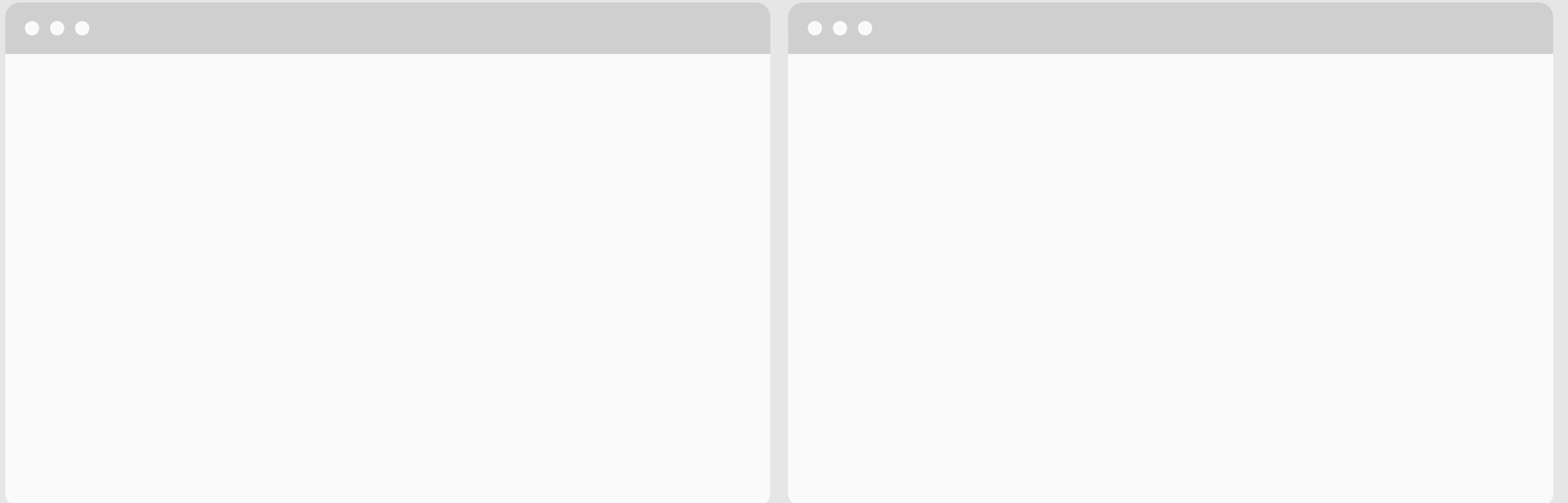
```
SELECT *  
FROM table1 t1  
WHERE LEFT(phone_number, 2) = '+7'
```

Вариант оптимизации

```
SELECT *  
FROM table1 t1  
WHERE phone_number LIKE '+7%';
```


Использование IN и альтернативы

Оператор **IN** может быть неэффективным для больших списков, так как не всегда использует индексы.



Использование IN и альтернативы

Оператор **IN** может быть неэффективным для больших списков, так как не всегда использует индексы.



```
SELECT *  
FROM my_table  
WHERE id IN (SELECT id FROM my_table2);
```


Использование IN и альтернативы

Оператор **IN** может быть неэффективным для больших списков, так как не всегда использует индексы.



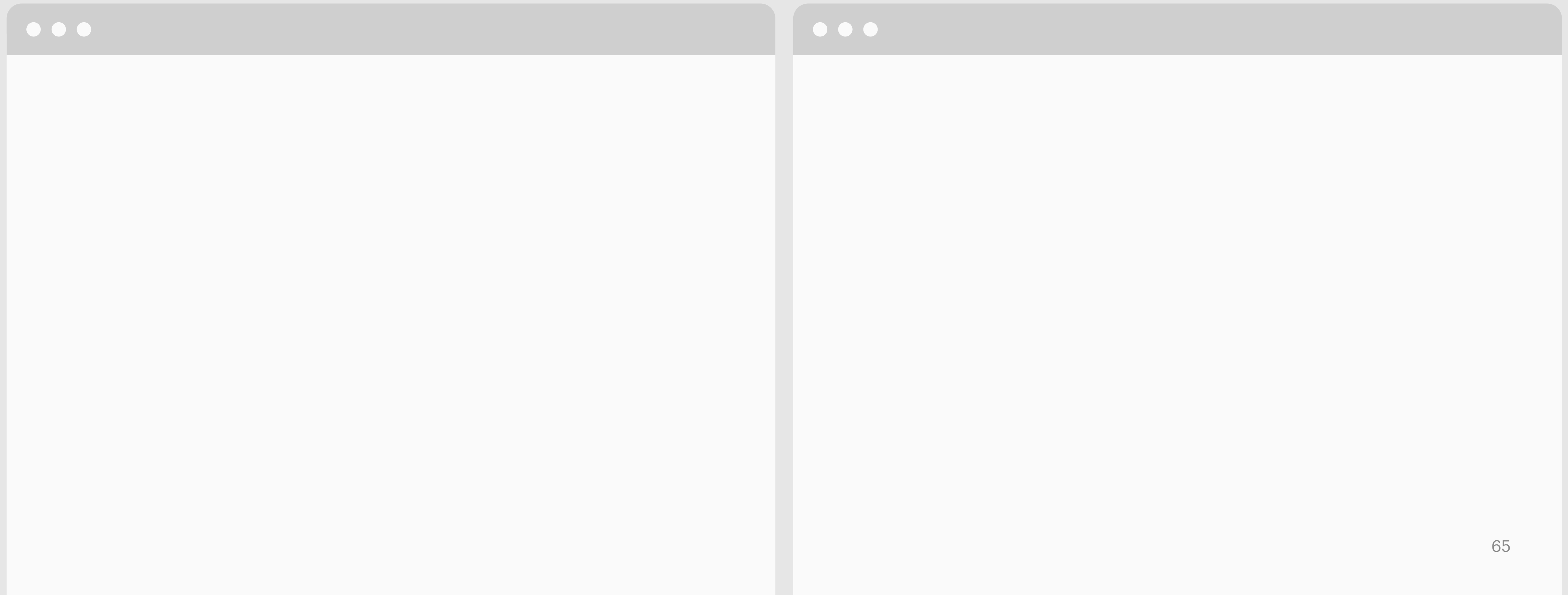
```
SELECT *  
FROM my_table  
WHERE id IN (SELECT id FROM my_table2);
```

Для больших списков используйте JOIN.

```
SELECT *  
FROM my_table t1  
    INNER JOIN my_table2 t2  
    ON t1.id = t2.id;
```

Неоптимальные подзапросы

Коррелированные подзапросы могут быть медленными, так как выполняются для каждой строки внешнего запроса.



Неоптимальные подзапросы

Коррелированные подзапросы могут быть медленными, так как выполняются для каждой строки внешнего запроса.



Неоптимальный запрос

```
SELECT *  
FROM orders o  
WHERE EXISTS (  
    SELECT 1  
    FROM customers c  
    WHERE c.id = o.customer_id  
AND c.status = 'active'  
);
```


Неоптимальные подзапросы

Коррелированные подзапросы могут быть медленными, так как выполняются для каждой строки внешнего запроса.



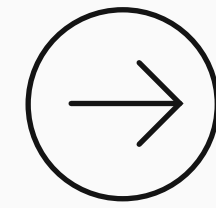
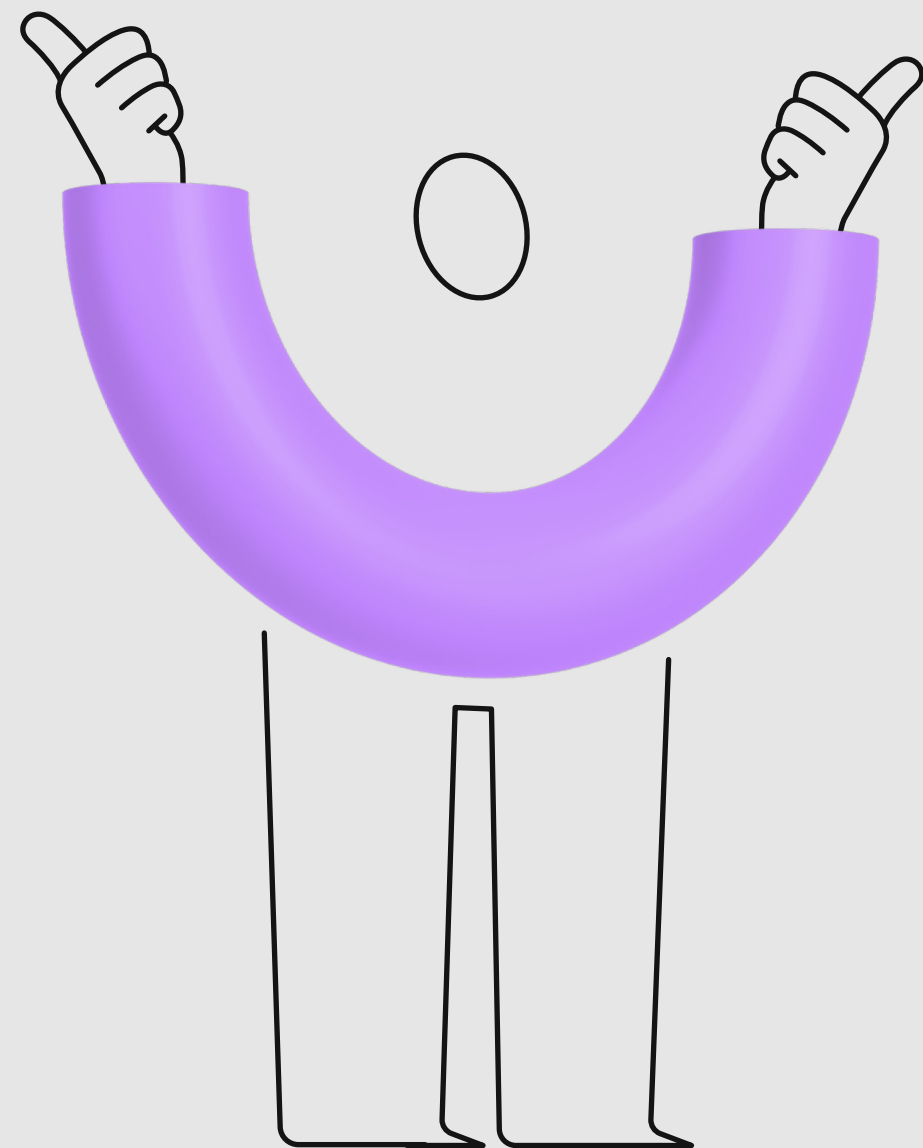
Неоптимальный запрос

```
SELECT *  
FROM orders o  
WHERE EXISTS (  
    SELECT 1  
    FROM customers c  
    WHERE c.id = o.customer_id  
AND c.status = 'active'  
);
```

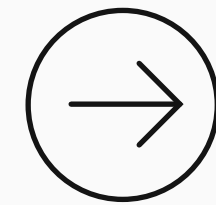
Вариант оптимизации

```
SELECT o.*  
FROM orders o  
    JOIN customers c  
    ON o.customer_id = c.id  
WHERE c.status = 'active';
```

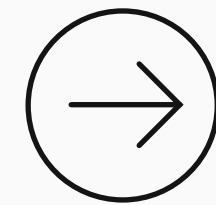
Резюме



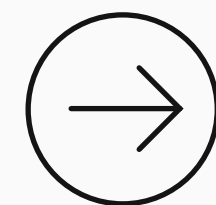
Планировщик формирует эффективный план, опираясь на статистику и факторы стоимости.



Инструменты **EXPLAIN** и **EXPLAIN ANALYZE** помогают выявить узкие места, оценить время выполнения и распределение ресурсов.



Партиционирование позволяет «разбить» таблицу на маленькие части, что способствует ускоренному обращению к данным.



Грамотное проектирование запросов — ключ к снижению нагрузки и ускорению обработки данных!

Вопросы

